CS260R: Reinforcement Learning

Mini Project Report

Student: Yingjie Huang | Agent: Delator | UID: 205927975

Abstract

To make a racing agent outperform others it must be both stable and fast across any map. This presents an inherent tradeoff: driving too aggressively risks crashes that bring speed to zero, while overly conservative behavior yields slow speed. I have done many experiments at the beginning of the project by only setting opponents to be hard opponents, adding rewards and recursively racing against self or modifying hyperparameters but the outcome is not promising. Instead, the training against previous-self trick advantage will diminish and hurt performance of the agent because of the high variance of the hard agent. PPO is stochastic so each rollout may have significant variances, making the agent hard to learn from scratch. As a result, the agent must start from an easy setting, otherwise it cannot learn a stable policy to evolve for future training.

To count for this, I first use weak model as opponents such as slow agent and still agent to train an agent that is familiar with the environment. Still and slow agent are stable since they can't move a lot in a rollout thus each time the agent we trained can actually learn the map instead of having to take the unstable opponents into consideration. Since the maps are fixed on the evaluation server, I manually create some maps like Serpentine and Oval maps on the server to make the new Baseline Agent represent a normal racer on the leaderboard. Then I train the agent from scratch and add some reward on overtake and speed and punishment on being out of the road to satisfy the requirements. As being out of the road is a loss, it must be punished. But throughout my previous experiments I find rewards can't be modified too drastically otherwise the agent will be displaying weird behaviors like not being afraid of crashing to sidewalk any more. So I decided to just slightly modify the built-in rewards to ensure the stability of the training.

Method

Environment

All experiments use MetaDrive's MultiAgentRacingEnv created by the VAIL lab at UCLA, running on Google Colab Pro.

Algorithm and Hyperparameters

Proximal Policy Optimization (PPO) from Stable-Baselines3. PPO is on policy. All hyperparameters inherit from the course staff except the seed was changed to 512 which I tested multiple times for getting the best seed so far and larger n steps to 2048 and corresponding batch size to 1024 following the tradition. The training is involved in two sequential setups: 1. Train a new BASELINE agent using still or self-created slow agent as opponent on circuit map. This converges fast since the setting is simple. I use 2 to 4 million timesteps for training this agent and I will pick the best among them. 2. Add the total four maps on the evaluation server and train an agent from scratch and add a small sparse overtake reward while setting the opponent to be the new BASELINE agent. This task is more complicated and I use about range from 5 to 6 million time steps for this task. Repeat for several rounds to get the best agent among the rounds.

Reward Engineering

The default MetaDrive reward plus a sparse overtake reward.

Training Efforts

Starting from default setting, I have first tried using diverse opponents such as adding the given example or baseline opponents to the opponent policy in env.py. However, together with the default opponents like still and aggressive opponents, too many opponents sometimes ruin the training process by adding too many variances at the beginning. And due to the frequent update of the policy in the original setup, which is of nstep of 512 and batch size of 256, the instability increases. I then switched to having a larger nstep and batch size, which stabilizes policy updates. Then I have tried adding my previous trained agent as opponent together with example and baseline agent and I iterate for several rounds. Although its performance increases in the first few runs, its behavior converges after some iterations and stop improving. I added more rewards on speed and overtakes, but its stability was heavily impacted and keeps crashing to the wall. I then add more penalties on the crashes but the agent is getting both conservative and more tending to crashing. Which indicates that modifying rewards violently won't help a lot in getting target behaviors.

The lesson learned from early approaches includes reducing variances at the early training, having smaller policy updating frequency and avoiding modifying too much on rewards. I then came to the idea to train an agent that can represent a simple opponent agent behavior on the evaluation server. I used still and self-created slow agent as opponents to train the new BASELINE agent that will be used as the opponent for future training. Slow and still agents display fixed behaviors so the variance is reduced for training this new agent. I retained a best agent among them and train another agent from scratch against it for longer time and more maps so it can learn the whole behavior needed for winning the race on the evaluation server.

Selected Experiments

Here are my efforts starting from course staff's default files.

Experiment	Opponent	Wins	Avg Route	Avg Speed	Notes
1. Default PPO	Example	1/5	58.67%	55.7 km/h	Poor performance
1. Default PPO	Baseline	1/5	75.14%	56.3 km/h	Lost 4/5 vs baseline agent
2. + Diverse Opponents	Example	4/5	96.21%	71.0 km/h	Strong improvement
2. + Diverse Opponents	Baseline	2/5	73.60%	60.8 km/h	Inconsistent episodes
3. + n_steps=2048, batch=1024	Example	4/5	91.98%	71.9 km/h	Faster, stabler updates

Experiment	Opponent	Wins	Avg Route	Avg Speed	Notes
3. + n_steps=2048, batch=1024	Baseline	3/5	94.20%	68.1 km/h	Best route completion
4. + Overtake Reward ×5	Example	1/5	88.19%	70.3 km/h	Faster speed but struggles to finish the race
4. + Overtake Reward ×5	Baseline	1/5	83.35%	65.7 km/h	Same problem as racing against example agent
5. batch=256, n_steps=512	Example	0/5	76.91%	64.7 km/h	Regression — small batch
5. batch=256, n_steps=512	Baseline	2/5	84.38%	56.6 km/h	Mixed results
6. batch=1024, n_steps=2048, reward on speed +0.001 diverse opponents	Example	0/5	78.86%	56.2 km/h	Worse performance
6. batch=1024, n_steps=2048, reward on speed +0.001 diverse opponents	Baseline	3/5	86.21%	63.1 km/h	Outperforms baseline easily
7. batch=1024, n_steps=2048, reward on speed +0.001 punish on crash -20 diverse opponents and 6 5.4 million steps	Example	1/5	89.28%	59.7 km/h	Improves on road finish but slow on speed
7. batch=1024, n_steps=2048, reward on speed punish on crash -20 diverse opponents and 6 5.4 million steps	Baseline	2/5	81.92%	51.9 km/h	Too conservative policy can't even beat baseline model
8. batch=1024, n_steps=2048, opponent=still 5.4 million steps Seed=512	Example	3/5	95.90%	72.0 km/h	Having a still opponent increases performance a lot.
8. batch=1024, n_steps=2048, opponent=still 5.4 million steps Seed=512	Baseline	3/5	86.20%	64.8 km/h	Sometimes the agent is blocked by baseline model.
9. batch=1024, n_steps=2048, seed=512	Example	4/5	99.70%	71.9 km/h	The agent becomes stable and fast now.

Experiment	Opponent	Wins	Avg Route	Avg Speed	Notes
opponent=8,reward on overtake +5 5.4 million steps					
9. batch=1024, n_steps=2048, seed=512 opponent=8,reward on overtake +5	Baseline	4/5	96.25%	68.8 km/h	Following same behavior as 8, sometimes it will crash into baseline agent at the beginning.

Experiment 1 — Default PPO (Course Staff Baseline)

vs. Example Agent

Ep	Route (Agent0)	Route (Agent1)	Speed (Agent0)	Speed (Agent1)	Winner / Notes
0	80.72%	57.98%	67.9 km/h	56.8 km/h	No finish
1	99.68%	66.08%	71.3 km/h	56.6 km/h	Arrived
2	5.38%	99.69%	45.0 km/h	67.0 km/h	Early crash
3	99.70%	99.67%	67.6 km/h	67.7 km/h	Arrived but loses
4	7.86%	99.68%	26.5 km/h	68.4 km/h	Early crash

Summary — avg reward 1065.33 | avg route 58.67% | avg speed 55.7 km/h | arrive 2/5 | wins 1/5

vs. Baseline Agent

Ep	Route (Agent0)	Route (Agent1)	Speed (Agent0)	Speed (Agent1)	Winner / Notes
0	99.77%	99.76%	73.0 km/h	73.1 km/h	agent1 wins (step 1064 vs 1067)
1	91.08%	99.67%	67.2 km/h	67.6 km/h	agent1 wins — agent0 DNF
2	99.74%	80.75%	69.8 km/h	66.1 km/h	agent0 wins (step 1118)
3	84.40%	99.66%	58.0 km/h	72.3 km/h	agent1 wins — agent0 DNF
4	0.71%	99.74%	13.5 km/h	56.4 km/h	agent1 wins — early crash

Agent0: | wins 1/5 | avg route 75.14% | avg speed 56.3 km/h | arrive 2/5

Experiment 2 — Diverse Opponents (example + baseline + original)

vs. Example Agent

Ep	Route (Agent0)	Route (Agent1)	Speed (Agent0)	Speed (Agent1)	Winner / Notes
0	99.68%	99.67%	74.4 km/h	71.9 km/h	agent0 wins (step 1048 vs 1078)
1	99.77%	99.67%	71.2 km/h	68.8 km/h	agent0 wins (step 1091 vs 1122)
2	82.23%	58.34%	65.7 km/h	61.5 km/h	No finish — both crash
3	99.67%	99.67%	72.2 km/h	70.7 km/h	agent0 wins (step 1078 vs 1095)
4	99.72%	99.76%	71.5 km/h	70.4 km/h	agent0 wins (step 1088 vs 1104)

Agent0: #1 | wins 4/5 | avg reward 1873.32 | avg route 96.21% | avg speed 71.0 km/h | arrive 4/5

vs. Baseline Agent

Ep	Route (Agent0)	Route (Agent1)	Speed (Agent0)	Speed (Agent1)	Winner / Notes
0	15.66%	15.49%	25.9 km/h	19.0 km/h	No finish — early crash
1	99.69%	70.71%	73.4 km/h	66.1 km/h	agent0 wins (step 1061)
2	99.69%	58.09%	70.3 km/h	52.7 km/h	agent0 wins (step 1104)
3	70.70%	13.74%	68.1 km/h	37.4 km/h	No finish
4	82.26%	69.55%	66.5 km/h	46.5 km/h	No finish

Agent0: #1 | wins 2/5 | avg reward 801.95 | avg route 73.60% | avg speed 60.8 km/h | arrive 2/5

Experiment 3 — Larger n_steps (2048) and Batch Size (1024)

vs. Example Agent

Ep	Route (Agent0)	Route (Agent1)	Speed (Agent0)	Speed (Agent1)	Winner / Notes
0	99.71%	99.76%	72.3 km/h	69.9 km/h	agent0 wins (step 1070 vs 1108)
1	99.66%	99.74%	72.8 km/h	69.5 km/h	agent0 wins (step 1064 vs 1114)
2	61.09%	61.12%	69.5 km/h	66.8 km/h	No finish — both crash
3	99.68%	91.11%	72.2 km/h	67.4 km/h	agent0 wins (step 1072)
4	99.73%	91.13%	72.6 km/h	68.2 km/h	agent0 wins (step 1065)

Agent0: #1 | wins 4/5 | avg reward 1664.73 | avg route 91.98% | avg speed 71.9 km/h | arrive 4/5

vs. Baseline Agent

Ep	Route (Agent0)	Route (Agent1)	Speed (Agent0)	Speed (Agent1)	Winner / Notes
0	91.12%	70.70%	67.7 km/h	66.7 km/h	No finish
1	99.70%	31.23%	71.2 km/h	56.4 km/h	agent0 wins (step 1090)
2	99.74%	84.52%	68.6 km/h	61.6 km/h	agent0 wins (step 1126)
3	99.74%	61.14%	67.6 km/h	67.4 km/h	agent0 wins (step 1145)
4	80.72%	99.74%	65.6 km/h	68.8 km/h	agent1 wins (step 1137)

Agent0: #1 | wins 3/5 | avg reward 1464.32 | avg route 94.20% | avg speed 68.1 km/h | arrive 3/5

Experiment 4 — Overtake Reward (+5) with Self-Play Opponent

vs. Example Agent

Ep	Route (Agent0)	Route (Agent1)	Speed (Agent0)	Speed (Agent1)	Winner / Notes
0	99.68%	99.74%	70.2 km/h	71.1 km/h	agent1 wins (step 1090 vs 1110)
1	61.15%	99.68%	69.1 km/h	72.1 km/h	agent1 wins — agent0 DNF
2	99.70%	99.67%	71.0 km/h	72.9 km/h	agent1 wins (step 1065 vs 1091)
3	80.73%	99.73%	68.4 km/h	72.2 km/h	agent1 wins — agent0 DNF
4	99.71%	66.74%	72.7 km/h	63.0 km/h	agent0 wins (step 1064)

Agent0: #2 | wins 1/5 | avg reward 1578.48 | avg route 88.19% | avg speed 70.3 km/h | arrive 3/5
Note: Overtake reward seems to help but not much.

vs. Baseline Agent

Ep	Route (Agent0)	Route (Agent1)	Speed (Agent0)	Speed (Agent1)	Winner / Notes
0	82.46%	82.24%	61.6 km/h	62.7 km/h	No finish
1	61.14%	99.71%	67.1 km/h	69.0 km/h	agent1 wins — agent0 DNF
2	91.07%	71.98%	68.4 km/h	63.5 km/h	No finish
3	82.37%	66.22%	62.3 km/h	39.8 km/h	No finish
4	99.70%	61.11%	68.9 km/h	69.3 km/h	agent0 wins (step 1125)

Agent0: #1 | wins 1/5 | avg reward 1583.09 | avg route 83.35% | avg speed 65.7 km/h | arrive 1/5

Experiment 5 — Small Batch (256) and Short n_steps (512)

vs. Example Agent

Ep	Route (Agent0)	Route (Agent1)	Speed (Agent0)	Speed (Agent1)	Winner / Notes
0	83.77%	99.66%	54.4 km/h	73.2 km/h	agent1 wins (step 1060)
1	66.72%	99.68%	63.8 km/h	72.0 km/h	agent1 wins (step 1077)
2	82.59%	99.67%	65.1 km/h	70.7 km/h	agent1 wins (step 1095)
3	80.71%	65.22%	70.6 km/h	62.3 km/h	No finish
4	70.75%	83.53%	69.7 km/h	53.8 km/h	No finish

Agent0: #2 | wins 0/5 | avg reward 1501.64 | avg route 76.91% | avg speed 64.7 km/h | arrive 0/5

vs. Baseline Agent

Ep	Route (Agent0)	Route (Agent1)	Speed (Agent0)	Speed (Agent1)	Winner / Notes
0	65.56%	99.70%	44.1 km/h	72.3 km/h	agent1 wins (step 1084)
1	99.72%	22.05%	70.3 km/h	55.5 km/h	agent0 wins (step 1106)
2	57.22%	57.40%	30.6 km/h	33.6 km/h	No finish
3	99.72%	99.71%	71.5 km/h	72.8 km/h	agent1 wins (step 1076 vs 1090)
4	99.69%	58.21%	66.6 km/h	48.3 km/h	agent0 wins (step 1163)

Agent0: #1(tie) | wins 2/5 | avg reward -113.96 | avg route 84.38% | avg speed 56.6 km/h | arrive 3/5
Note: Small n steps cause high variance and diminish the stability of PPO.

Experiment 6 — Diverse Opponents + Reward on Speed x 0.001

vs. Example Agent

Ep	Route (Agent0)	Route (Agent1)	Speed (Agent0)	Speed (Agent1)	Winner / Notes
0	91.07%	99.73%	67.6 km/h	73.1 km/h	Agent 1
1	57.58%	99.71%	45.1 km/h	71.5 km/h	Agent 1
2	82.74%	80.73%	35.4 km/h	39.5 km/h	No finish — both crash
3	80.70%	99.73%	68.2 km/h	71.3 km/h	Agent 1
4	82.20%	99.73%	64.9 km/h	70.5 km/h	Agent 1

agent0 (agent_205927975): wins=0, losses=5, avg_reward=-463.63, avg_route=78.86%, avg_speed=56.2 km/h, arrive=0/5

vs. Baseline Agent

Ep	Route (Agent0)	Route (Agent1)	Speed (Agent0)	Speed (Agent1)	Winner / Notes
0	99.75%	84.07%	25.9 km/h	50.6 km/h	Agent 0
1	61.16%	71.51%	73.4 km/h	60.0 km/h	None
2	99.76%	99.69%	70.3 km/h	66.9 km/h	Agent 0
3	99.65%	64.30%	68.1 km/h	45.7 km/h	Agent 0
4	70.72%	0.74%	66.5 km/h	7.3 km/h	None

agent0 (agent_205927975): wins=3, losses=2, avg_reward=905.48, avg_route=86.21%, avg_speed=63.1 km/h, arrive=3/5

Experiment 7 — Diverse Opponents + Reward on Speed x 0.001 + Penalty Crash x 20

vs. Example Agent

Ep	Route (Agent0)	Route (Agent1)	Speed (Agent0)	Speed (Agent1)	Winner / Notes
0	99.73%	80.68%	54.1 km/h	53.7 km/h	Agent 0
1	99.76%	99.74%	62.9 km/h	70.8 km/h	Agent 1
2	82.21%	99.76%	64.3 km/h	71.3 km/h	Agent 1
3	64.95%	99.72%	50.7 km/h	68.4 km/h	Agent 1
4	99.74%	99.75%	66.5 km/h	71.1 km/h	Agent 1

agent0 (agent_205927975): wins=1, losses=4, avg_reward=1588.76, avg_route=89.28%, avg_speed=59.7 km/h, arrive=3/5

vs. Baseline Agent

Ep	Route (Agent0)	Route (Agent1)	Speed (Agent0)	Speed (Agent1)	Winner / Notes
0	99.73%	99.70%	59.9 km/h	72.1 km/h	Agent 1
1	99.73%	99.76%	56.3 km/h	67.8 km/h	Agent 1
2	10.74%	66.40%	23.5 km/h	35.6 km/h	None
3	99.68%	82.52%	59.7 km/h	65.9 km/h	Agent 0
4	99.73%	54.23%	60.2 km/h	68.6 km/h	Agent 0

agent0 (agent_205927975): wins=2, losses=3, avg_reward=502.48, avg_route=81.92%, avg_speed=51.9 km/h

Experiment 8 — Still Opponent 5.4 million steps Seed=512

vs. Example Agent

Ep	Route (Agent0)	Route (Agent1)	Speed (Agent0)	Speed (Agent1)	Winner / Notes
0	99.72%	99.68%	74.2 km/h	71.7 km/h	Agent 0
1	99.67%	99.76%	74.3 km/h	70.9 km/h	Agent 0
2	99.71%	99.74%	71.4 km/h	71.5 km/h	Agent 1
3	80.68%	70.71%	68.7 km/h	69.8 km/h	None
4	99.73%	70.74%	71.5 km/h	69.3 km/h	Agent 0

agent0 (agent_205927975): wins=3, losses=2, avg_reward=873.67, avg_route=95.90%, avg_speed=72.0 km/h, arrive=4/5

vs. Baseline Agent

Ep	Route (Agent0)	Route (Agent1)	Speed (Agent0)	Speed (Agent1)	Winner / Notes
0	70.77%	80.71%	70.2 km/h	64.7 km/h	None
1	99.74%	57.41%	42.5 km/h	34.8 km/h	Agent 0
2	99.66%	67.01%	70.0 km/h	35.3 km/h	Agent 0
3	99.75%	61.10%	73.0 km/h	60.1 km/h	Agent 0
4	61.10%	31.27%	68.4 km/h	62.5 km/h	None

agent0 (agent_205927975): wins=3, losses=2, avg_reward=598.14, avg_route=86.20%, avg_speed=64.8 km/h, arrive=3/5

Experiment 9 — Opponent from exp 8, seed =512, multiple maps, +5 overtake reward, 5.4 million steps

vs. Example Agent

Ep	Route (Agent0)	Route (Agent1)	Speed (Agent0)	Speed (Agent1)	Winner / Notes
0	99.73%	99.68%	73.7 km/h	69.9 km/h	Agent 0
1	99.73%	99.74%	68.4 km/h	71.6 km/h	Agent 1
2	99.67%	99.65%	72.1 km/h	71.3 km/h	Agent 0
3	99.66%	99.70%	72.5 km/h	71.1 km/h	Agent 0
4	99.69%	70.69%	72.8 km/h	67.3 km/h	Agent 0

agent0 (agent_205927975): wins=4, losses=1, avg_reward=1512.03, avg_route=99.70%, avg_speed=71.9 km/h, arrive=5/5

vs. Baseline Agent

Ep	Route (Agent0)	Route (Agent1)	Speed (Agent0)	Speed (Agent1)	Winner / Notes
0	99.72%	12.79%	70.2 km/h	50.7 km/h	Agent 0
1	99.72%	91.70%	42.5 km/h	47.6 km/h	Agent 0
2	82.34%	67.55%	70.0 km/h	57.1 km/h	None
3	99.77%	12.83%	73.0 km/h	38.2 km/h	Agent 0
4	99.72%	57.76%	68.4 km/h	28.8 km/h	Agent 0

agent0 (agent_205927975): wins=4, losses=1, avg_reward=721.64, avg_route=96.25%, avg_speed=68.8 km/h, arrive=4/5

Analysis

What Improved Performance

- Racing against simple opponents help the agent stabilizes behavior and earns more rewards since it's trained on a low variance behavior.
- Larger nstep and batch sizes help stabilize the policy. Since the PPO already clips the changes, updating it too frequently will diminish PPO's clipping advantage by having too many updates on the policy.
- Running against the opponent trained with simple setups can stabilize training a lot.

What Hurt Performance

- Adding too many rewards or punishments will hurt the behavior stability and will not reach the destined behavior.
- Racing against too diverse opponents will add too many variances and noises to the training. The agent struggles on learning.
- Training for too many steps can't improve since the policy already converges in the early steps.

Key Insight

The most surprising finding was that in order to train a racing agent, we shouldn't aim to set hard agents as the opponent at the beginning. Instead, we need to do some steps to stabilize the opponent so that the learner can learn in a fixed setting.

Conclusion

To summarize, training a racing agent emphasizes more on finishing and stability. Adding rewards on speeding or punishing on crashing will not help the agents learn a stable racing behavior. Instead, it either adds too many noises to the policy or making the policy too conservative. Although the opponents may vary, their behaviors may converge at some point and that's how other agents on the evaluation server will also behave since all agents wish to finish the race and reach the destination. An agent trying to block or crash into other agents will result in a tie at best, which thus will not be the optimal strategy for a racing agent. So to ensure stability, training an agent representing a normal racing agent. It has fixed behavior aiming to finish the race as fast as possible and racing against it will thus help the agent getting used to the competition environment in the evaluation server.